



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第八章 图



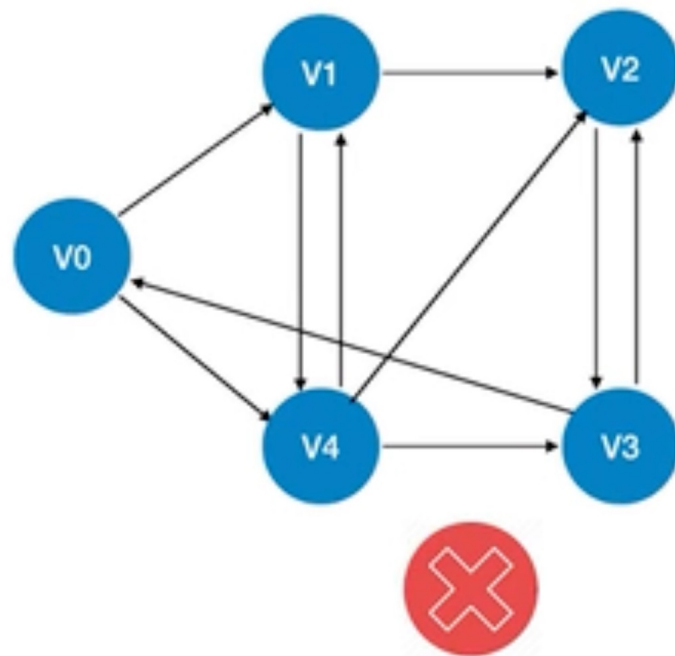
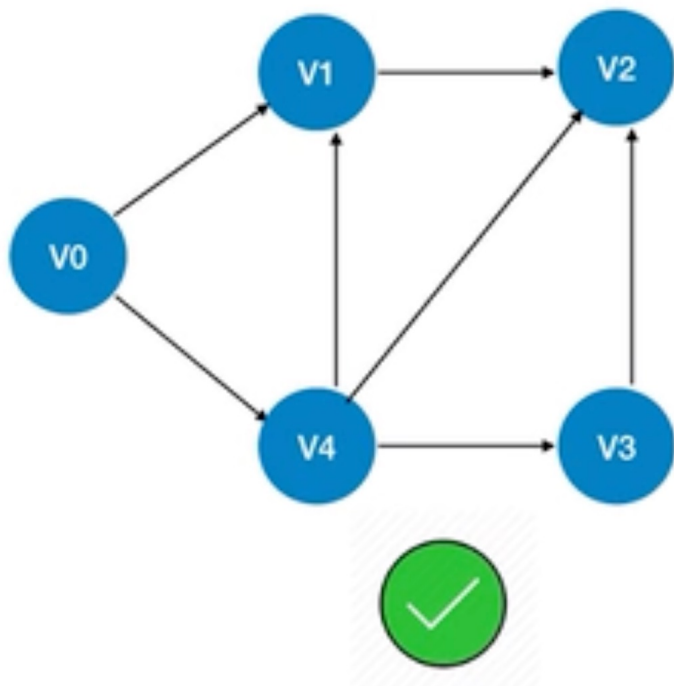
- 图的基本概念
- 图的存储结构
- 图的遍历与连通性
- 最小生成树
- 最短路径
- 活动网络



DAG图



有向无环图：若一个有向图中不存在环，则称为有向无环图，简称DAG图 (Directed Acyclic Graph)

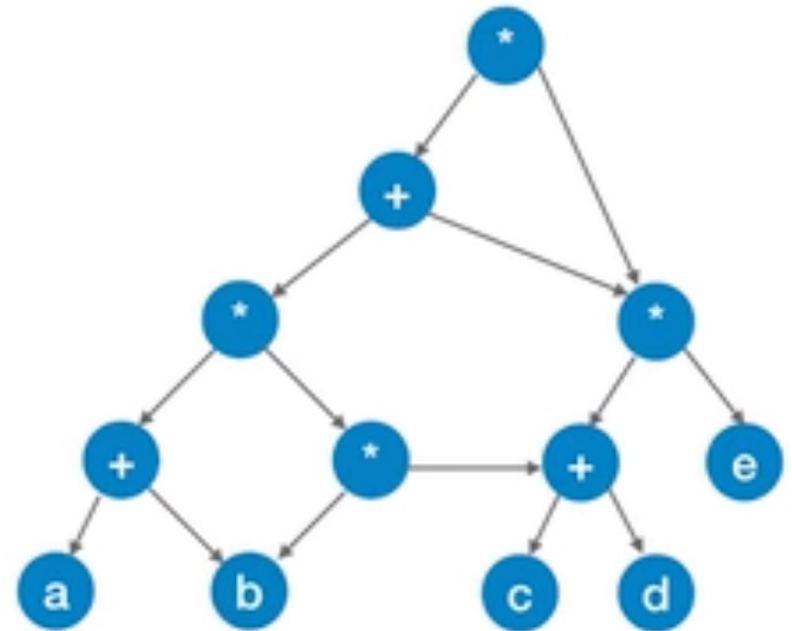
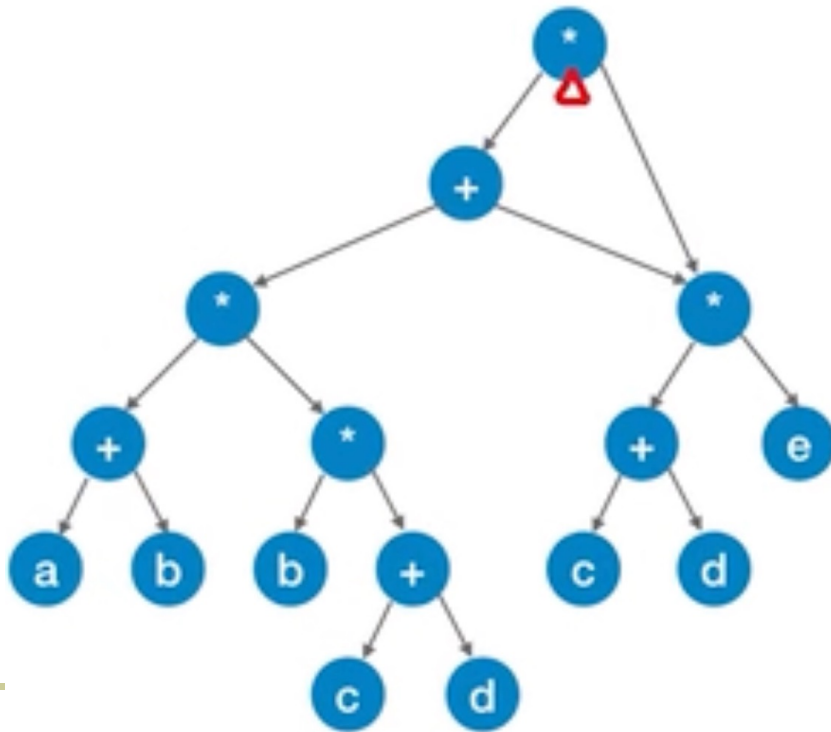




DAG示例



$$((a + b) * (b * (c + d)) + (c + d) * e) * ((c + d) * e)$$





活动网络 (Activity Network)



用顶点表示活动的网络

(activity on vertices, 即AOV网络)

- 计划、施工过程、生产流程、程序流程等都是“**工程**”。除了很小的工程外，一般都把工程分为若干个叫做“**活动**”的子工程。完成了这些活动，这个工程就可以结束了。
- 例如，计算机专业学生的学习就是一个工程，每一门课程的学习就是整个工程的一些活动。其中有些课程要求先修课程，有些则不要求。有的课程之间有领先关系，有的课程可以并行地学习。



课程代号

课程名称

先修课程



C₁

高等数学

C₂

程序设计基

C₃

离散数学

C₄

数据结构

C₅

高级语言程序设

C₆

编译方法

C₇

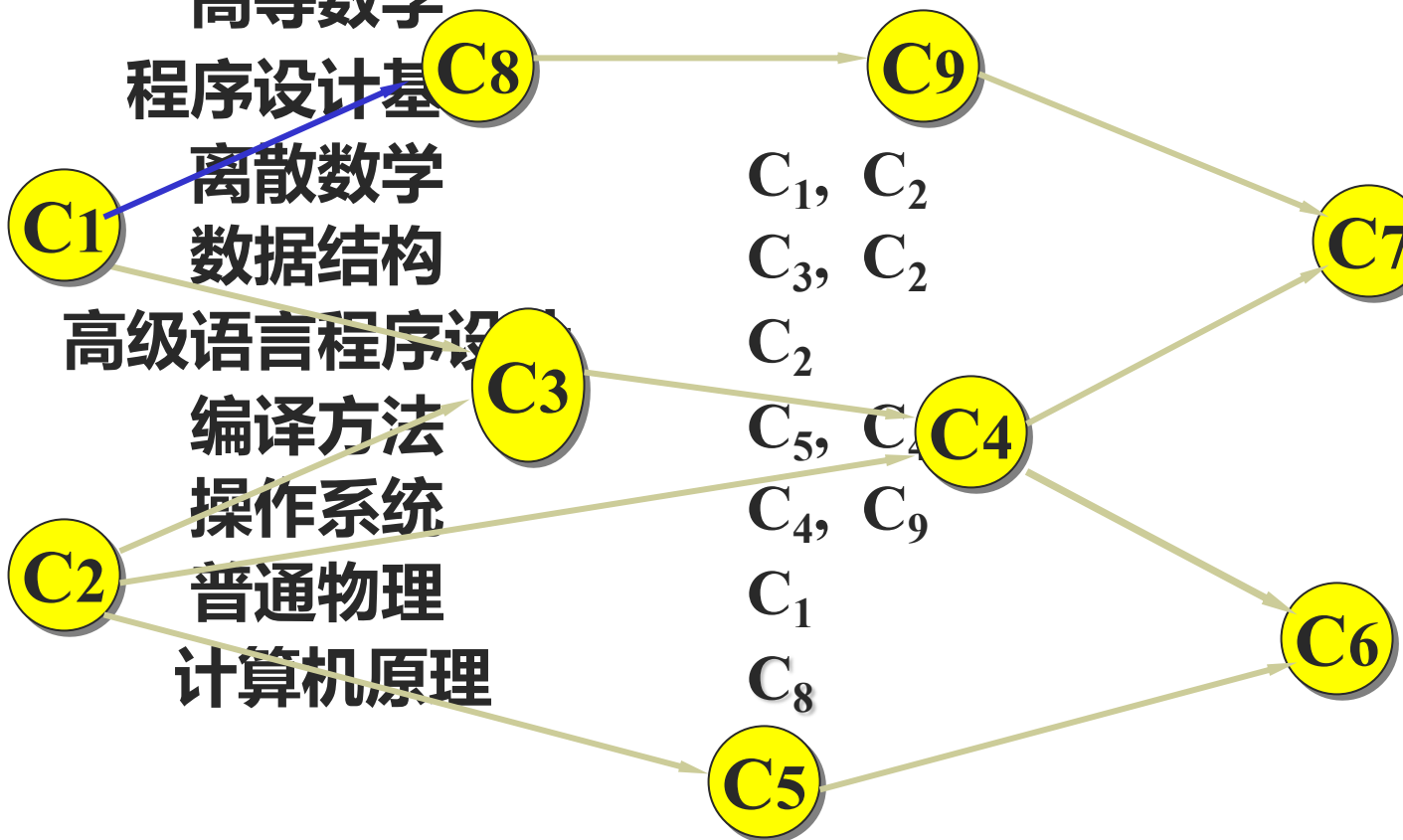
操作系统

C₈

普通物理

C₉

计算机原理



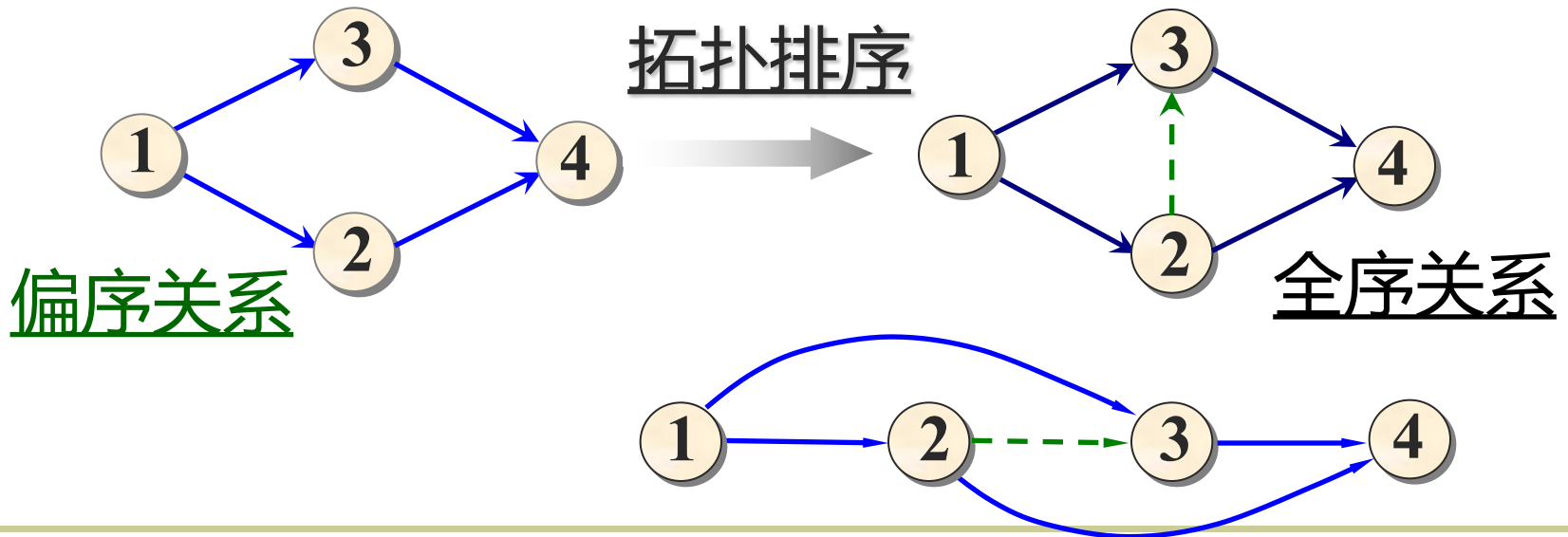
学生课程学习工程图



- 可以用有向图表示一个工程。在这种有向图中，用顶点表示活动，用有向边 $\langle V_i, V_j \rangle$ 表示活动 V_i 必须先于活动 V_j 进行。这种有向图叫做顶点表示活动的AOV网络 (Activity On Vertices)。
- 在AOV网络中不能出现有向回路，即有向环。如果出现了有向环，则意味着某项活动应以自己作为先决条件。
- 因此，对给定的AOV网络，必须先判断它是否存在有向环。



- 检测有向环的一种方法是对AOV网络构造它的**拓扑有序序列**。即将各个顶点 (代表各个活动)排列成一个线性有序的序列，使得AOV网络中所有应存在的前驱和后继关系都能得到满足。





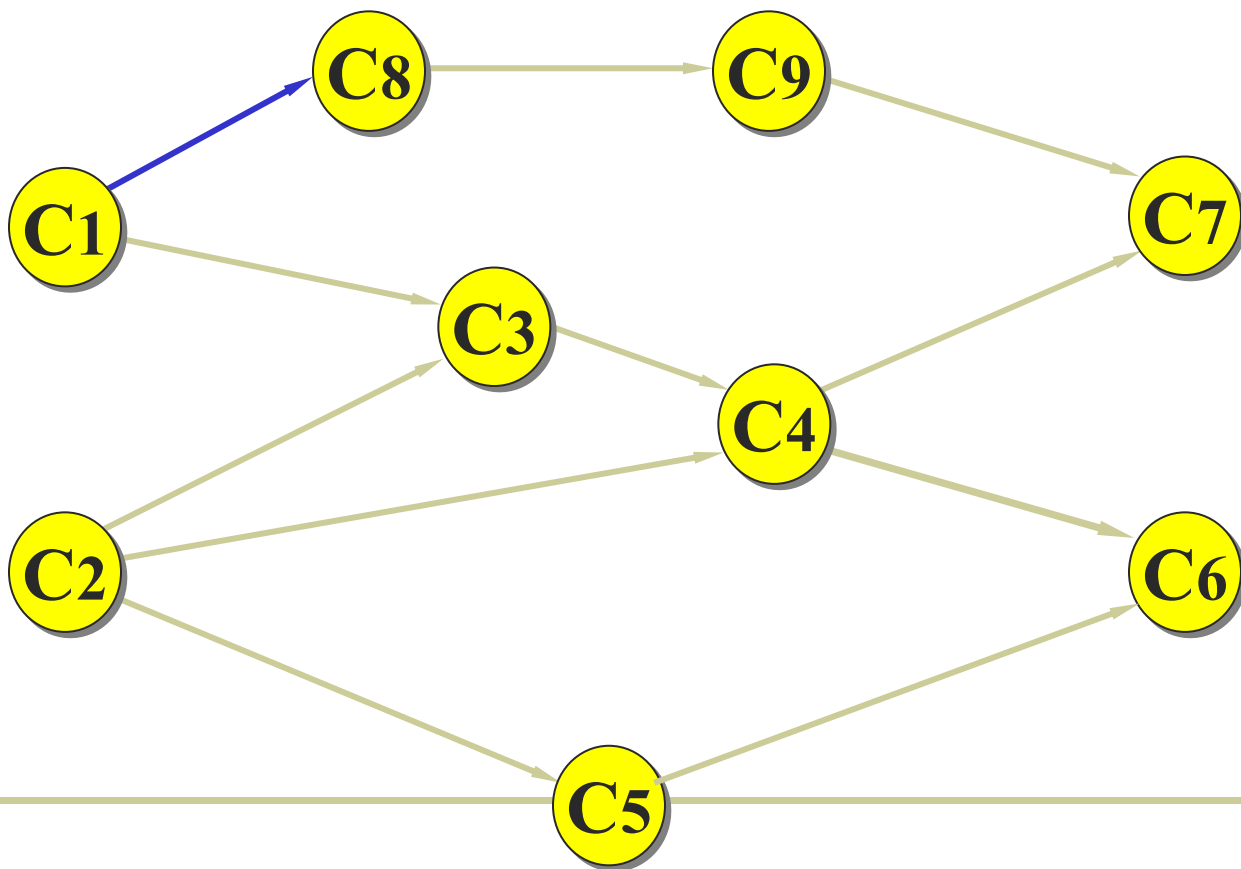
- 这种构造AOV网络全部顶点的拓扑有序序列的运算就叫做拓扑排序。
- 如果通过拓扑排序能将AOV网络的**所有顶点**都排入一个拓扑有序的序列中, 则该网络中必定不会出现有向环。
- 如果AOV网络中存在有向环, 此AOV网络所代表的工程是不可行的。



例如, 对学生选课工程图进行拓扑排序, 得到的
拓扑有序序列为:



$C_1, C_2, C_3, C_4, C_5, C_6, C_8, C_9, C_7$
或 $C_1, C_8, C_9, C_2, C_5, C_3, C_4, C_7, C_6$





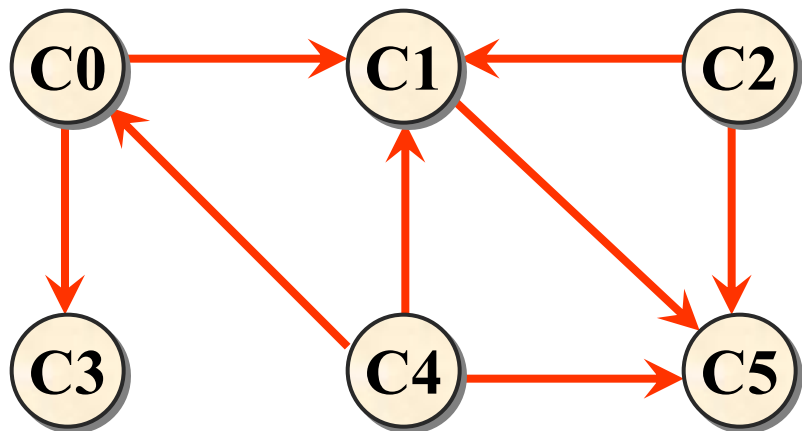
进行拓扑排序的方法



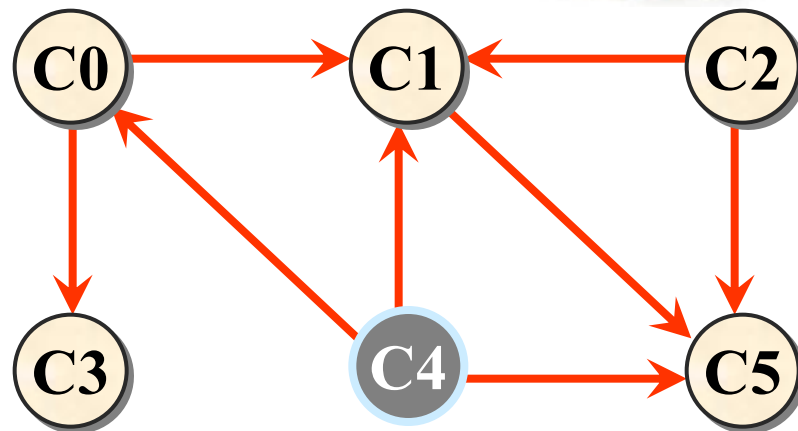
- ① 输入AOV网络, 令 n 为顶点个数。
- ② 在AOV网络中选一个没有直接前驱的顶点, 并输出之;
- ③ 从图中删去该顶点, 同时删去所有它发出的有向边;
- ④ 重复以上 ②、③步, 直到
 - ◆ 全部顶点均已输出, 拓扑有序序列形成, 拓扑排序完成;
 - ◆ 或图中还有未输出的顶点, 但已跳出处理循环, 说明图中还剩一些顶点, 它们都有直接前驱。这时网络中必存在有向环。



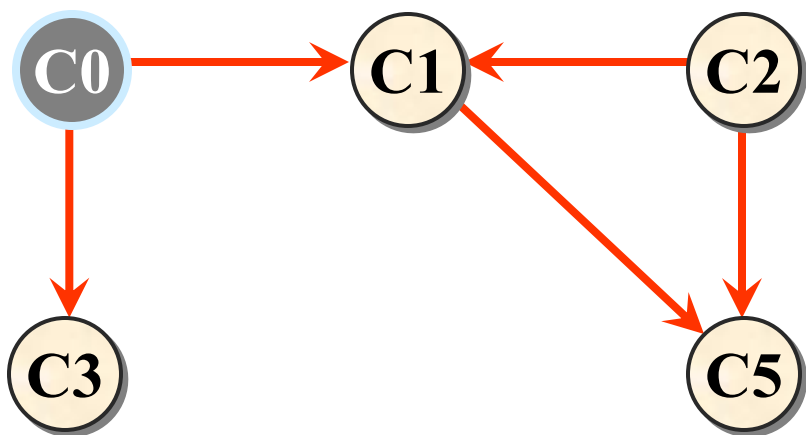
拓扑排序的过程



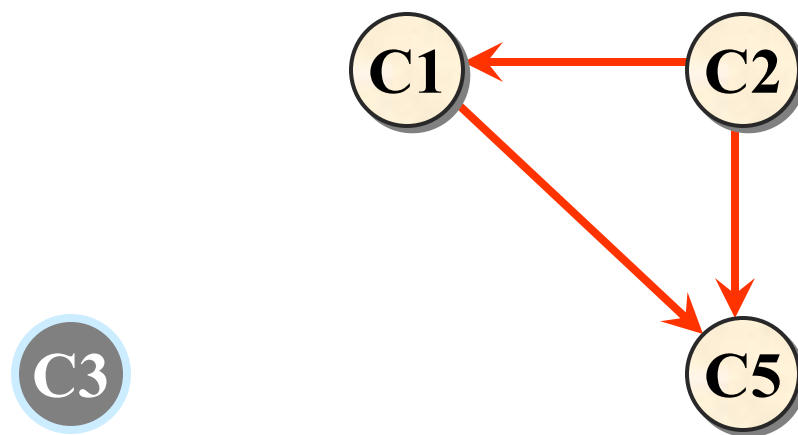
(a) 有向无环图



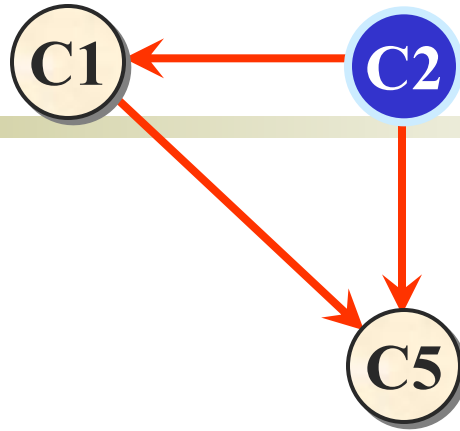
(b) 输出顶点C4



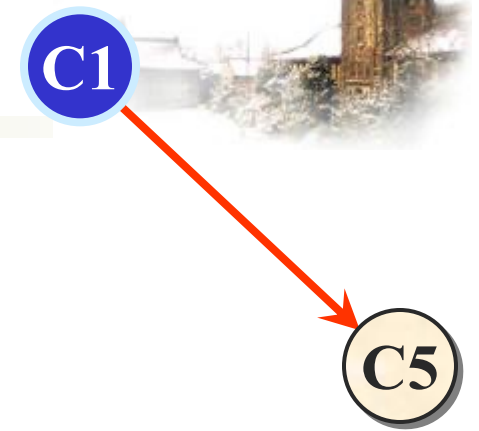
(c) 输出顶点C0



(d) 输出顶点C3



(e) 输出顶点C2



(f) 输出顶点C1



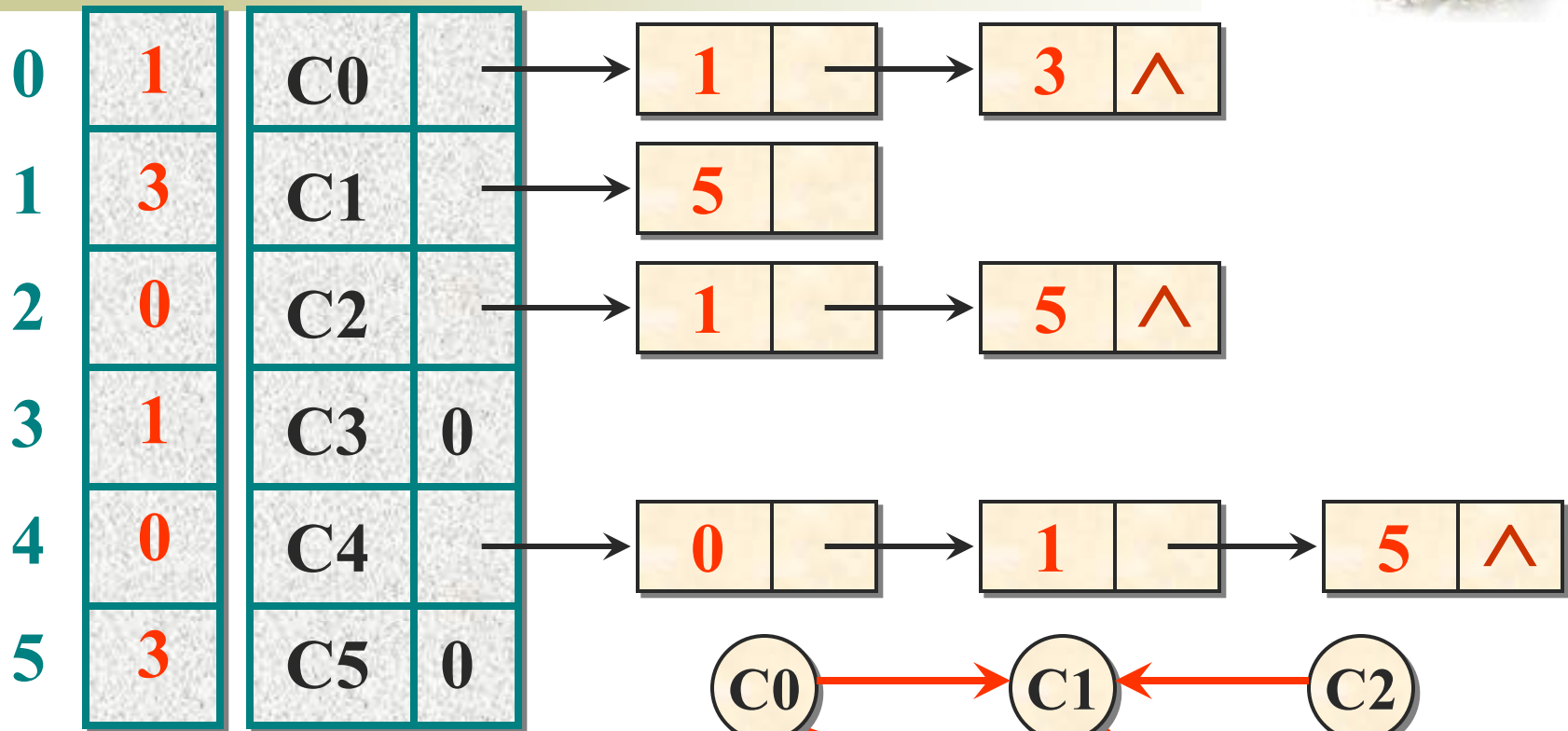
(g) 输出顶点C5

(h) 拓扑排序完成

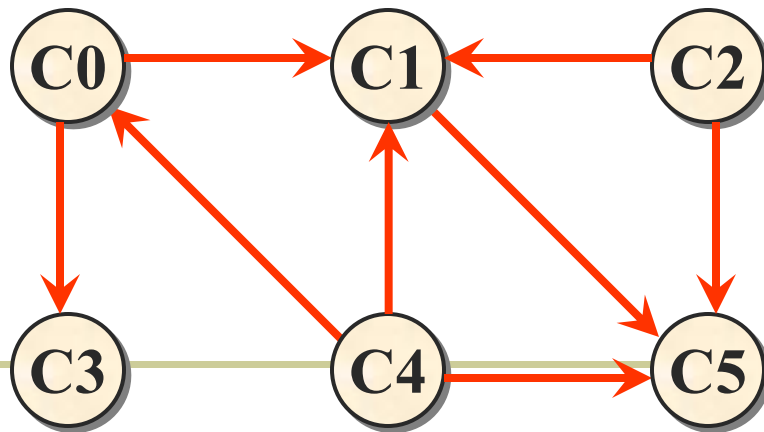
- 最后得到的拓扑有序序列为 $C_4, C_0, C_3, C_2, C_1, C_5$ 。它满足图中给出的所有前驱和后继关系，对于本来没有这种关系的顶点，如 C_4 和 C_2 ，也排出了先后次序关系。



count data adj dest link



AOV网络及其
邻接表表示





- 在邻接表中增设一个数组count[], 记录各顶点入度, 入度为零的顶点即无前驱顶点。
- 在输入数据前, 顶点表NodeTable[]和入度数组count[]全部初始化。在输入数据时, 每输入一条边<j, k>, 就需要建立一个边结点, 并将它链入相应边链表中, 统计入度信息:

```
Edge * p = new Edge<int> (k);
```

```
    //建立边结点, dest 域赋为 k
```

```
p->link = NodeTable[j].adj;
```

```
NodeTable[j].adj = p;
```

```
    //链入顶点j的边链表的前端
```

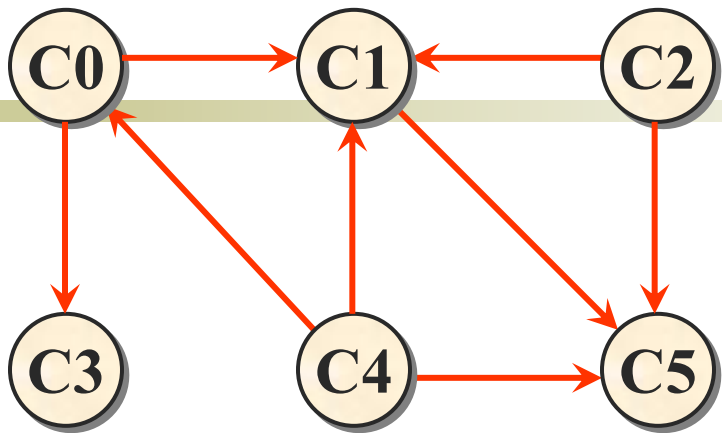
```
count[k]++;           //顶点k入度加一
```



- 在算法中, 使用一个存放入度为零的顶点的链式栈, 供选择和输出无前驱的顶点。
- 拓扑排序算法可描述如下:
 - ◆ 建立入度为零的顶点栈;
 - ◆ 当入度为零的顶点栈不空时, 重复执行
 - ✿ 从顶点栈中退出一个顶点, 并输出之;
 - ✿ 从AOV网络中删去这个顶点和它发出的边, 边的终顶点入度减1;
 - ✿ 如果边的终顶点入度减至0, 则该顶点进入度为零的顶点栈;
 - ◆ 如果输出顶点个数少于AOV网络的顶点个数, 则报告网络中存在有向环。



- 在算法实现时, 为了建立入度为0的顶点栈, 可以不另外分配存储空间, 直接利用入度为0的顶点的count[]数组元素。设立一个栈顶指针top, 指示当前栈顶位置, 即某一个入度为0的顶点。栈初始化时置 $top = -1$ 。
- 将顶点i 进栈时执行以下指针的修改:
 $count[i] = top; top = i;$
// top指向新栈顶i, 原栈顶元素序号在count[i]中
- 退栈操作可以写成:
 $j = top; top = count[top];$
//位于栈顶的顶点记于 j, top退到次栈顶



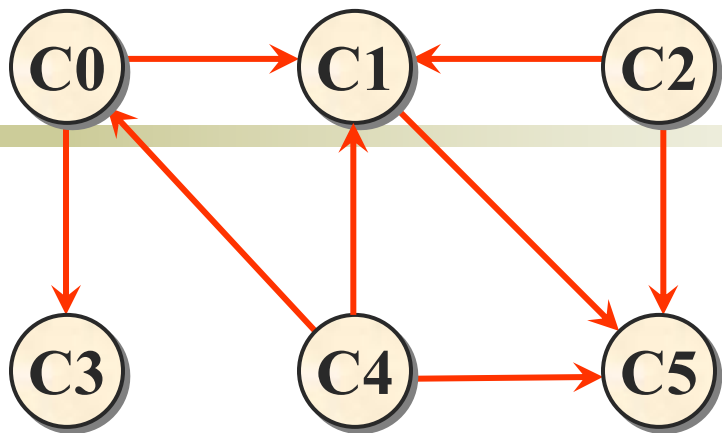
拓扑排序时入度为0的顶点栈在count[]中的变化

top→	0	1
	1	3
	2	0
	3	1
	4	0
	5	3

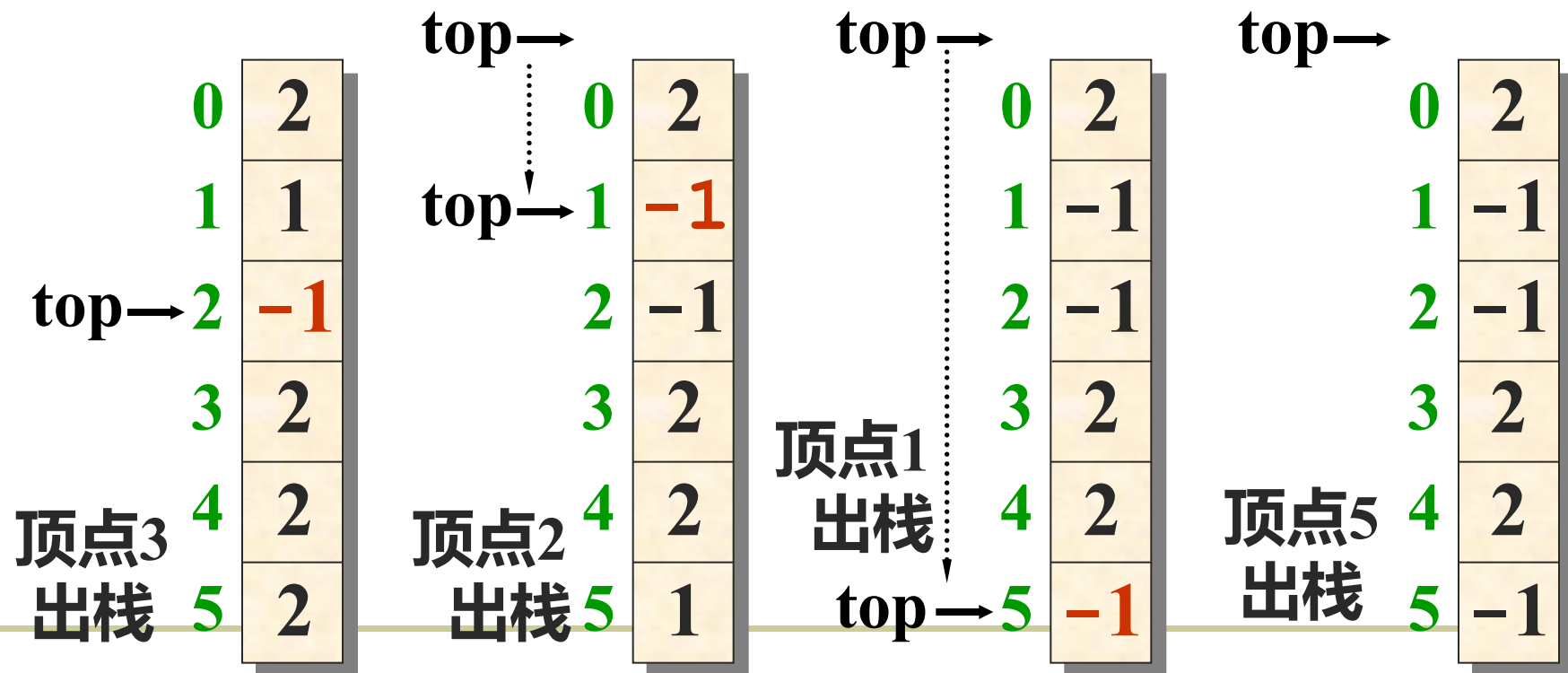
top→	0	1
	1	3
top→	2	-1
	3	1
top→	4	2
建栈	5	3

top→	0	2
	1	2
top→	2	-1
	3	1
顶点4	4	2
出栈	5	2

	0	2
	1	1
top→	2	-1
top→	3	2
顶点0	4	2
出栈	5	2



拓扑排序时入度为0的顶点栈在count[]中的变化





拓扑排序的算法



```
template <class T, class E>
void TopologicalSort (Graph<T, E>& G) {
    int i, j, w, v;
    int top = -1;           //入度为0的顶点栈初始化
    int n = G.NumberOfVertices(); //网络中顶点个数
    int *count = new int[n];
                           //入度数组兼入度为0的顶点栈
    for (i = 0; i < n; i++) count[i] = 0;
    cin >> i >> j;        //输入一条边(i, j)
    while (i > -1 && i < n && j > -1 && j < n) {
        G.insertEdge (i, j); count[j]++;
    }
}
```



```
    cin >> i >> j;
}
for (i = 0; i < n; i++)    //检查网络所有顶点
    if (count[i] == 0)    //入度为0的顶点进栈
        { count[i] = top; top = i; }
for (i = 0; i < n; i++)    //期望输出n个顶点
    if (top == -1) {    //中途栈空, 转出
        cout << "网络中有回路! " << endl;
        return;
    }
    else {    //继续拓扑排序
        v = top; top = count[top];    //退栈v
```



```
cout << G.getValue(v) << " " << endl; //输出
w = G.GetFirstNeighbor(v);
while (w != -1) { //扫描顶点v的出边表
    count[w]--; //邻接顶点入度减1
    if (count[w]==0) //入度减至0, 进栈
        { count[w] = top; top = w; }
    w = G.GetNextNeighbor (v, w);
} //一个顶点输出后, 调整其邻接顶点入度
//输出一个顶点, 继续for循环
};
```



- 分析此拓扑排序算法可知，如果AOV网络有 n 个顶点、 e 条边，在拓扑排序的过程中，搜索入度为0的顶点，建立链式栈所需要的时间是 $O(n)$ 。
- 在正常的情况下
 - 有向图有 n 个顶点，每个顶点进一次栈，出一次栈，共输出 n 次。
 - 顶点入度减1的运算共执行了 e 次。
- 所以，总的时间复杂度为 $O(n+e)$ 。



用边表示活动的网络(AOE网络)



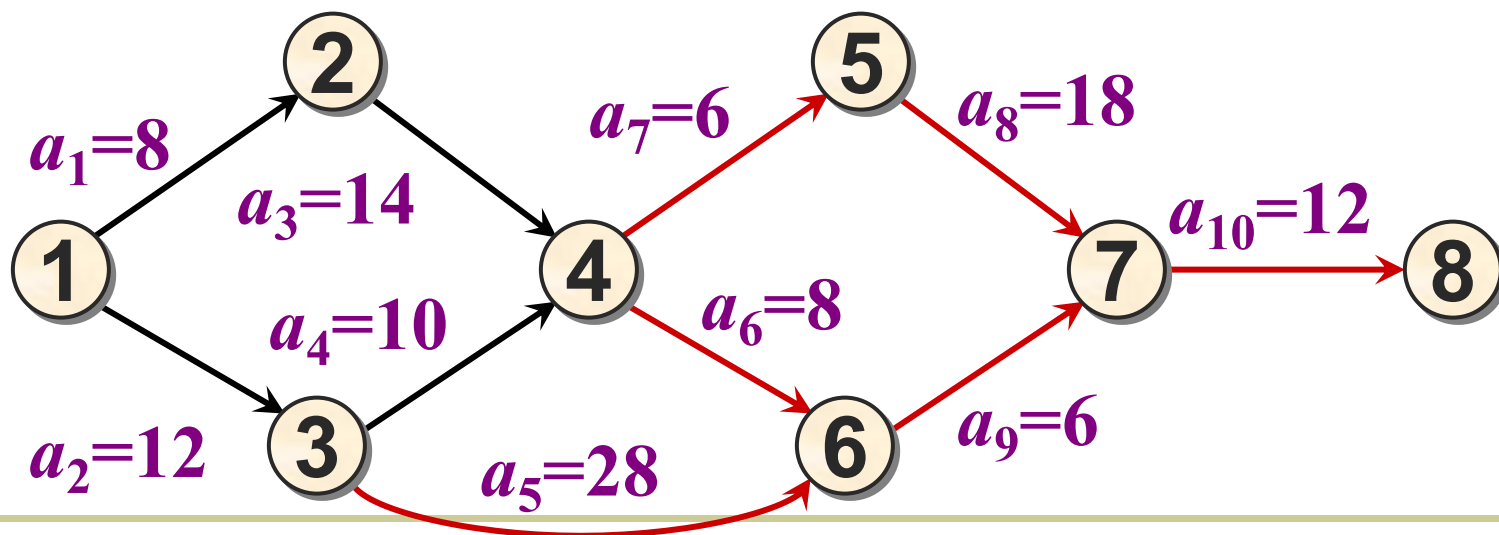
- 如果在无有向环的带权有向图中, 用有向边表示一个工程中的活动 (Activity), 用边上权值表示活动持续时间 (Duration), 用顶点表示事件 (Event), 则这样的有向图叫做**用边表示活动的网络**, 简称 AOE (Activity On Edges) 网络。
- AOE网络在某些工程估算方面非常有用。例如, 可以使人们了解:
 - ◆ 完成整个工程至少需要多少时间(假设网络中没有环)?
 - ◆ 为缩短完成工程所需的时间, 应当加快哪些活动?



- 整个工程只有一个开始点和一个完成点，开始点（即入度为零的顶点）称为**源点**，结束点（即出度为零的顶点）称为**汇点**。
- 从源点到各个顶点、以及从**源点**到**汇点**的有向路径可能不止一条。这些路径的长度也可能不同。完成不同路径的活动所需的时间虽然不同，但只有各条路径上所有活动都完成了，整个工程才算完成。
- 因此，**完成整个工程所需的时间取决于从源点到汇点的最长路径长度**，即在这条路径上所有活动的持续时间之和。**这条路径长度最长的路径就叫做关键路径**(Critical Path)。



- 要找出关键路径，必须找出关键活动，即不按期完成就会影响整个工程完成的活动。
- 关键路径上的所有活动都是关键活动。因此，只要找到了关键活动，就可以找到关键路径。
- 例如，下图是一个AOE网。





定义几个与计算关键活动有关的量:



1. 事件 V_i 的**最早可能开始时间** $Ve(i)$:
是从源点 V_0 到顶点 V_i 的**最长路径长度**。
2. 事件 V_i 的**最迟允许开始时间** $VL[i]$:
是在保证汇点 V_{n-1} 在 $Ve[n-1]$ 时刻完成的前提下,
事件 V_i 的允许的最迟开始时间。
3. 活动 a_k 的**最早可能开始时间** $Ae[k]$:
设活动 a_k 在边 $\langle V_i, V_j \rangle$ 上, 则 $Ae[k]$ 是从源点 V_0 到 顶
点 V_i 的最长路径长度。因此,
$$Ae[k] = Ve[i].$$



4. 活动 a_k 的最迟允许开始时间 $Al[k]$:

$Al[k]$ 是在不会引起时间延误的前提下, 该活动允许的最迟开始时间。

$$Al[k] = Vl[j] - dur(<i, j>).$$

其中, $dur(<i, j>)$ 是完成 a_k 所需的时间。

5. 时间余量 $Al[k] - Ae[k]$:

表示活动 a_k 的最早可能开始时间和最迟允许开始时间的的时间余量。 $Al[k] == Ae[k]$ 表示活动 a_k 是没有时间余量的关键活动。

- 为找出关键活动, 需要求各个活动的 $Ae[k]$ 与 $Al[k]$, 以判别是否 $Al[k] == Ae[k]$ 。

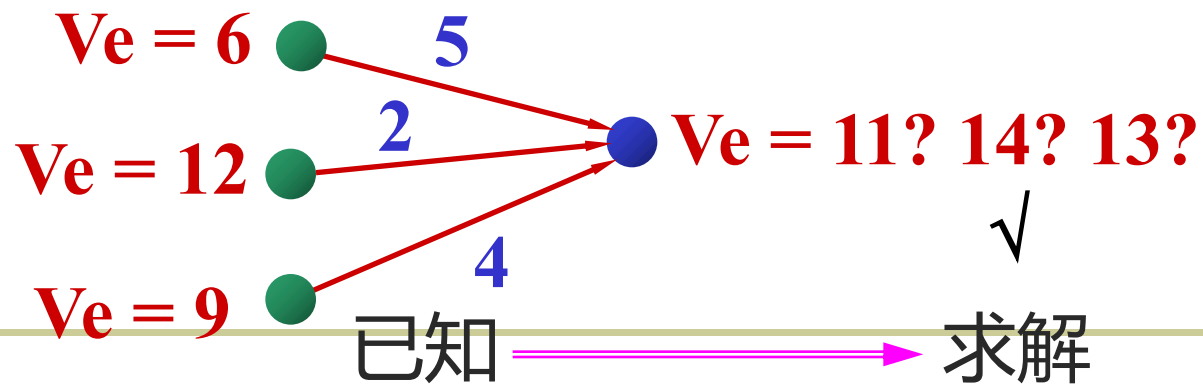


- 为求得 $Ae[k]$ 与 $Al[k]$, 需要先求得从源点 V_0 到各个顶点 V_i 的 $Ve[i]$ 和 $Vl[i]$ 。
- 求 $Ve[i]$ 的递推公式
 - ◆ 从 $Ve[0] = 0$ 开始, 向前递推

$$Ve[j] = \max \{ Ve[i] + dur(<V_i, V_j>) \},$$

$<V_i, V_j> \in S2, j = 1, 2, \dots, n-1$

$S2$ 是所有指向 V_j 的有向边 $<V_i, V_j>$ 的集合。



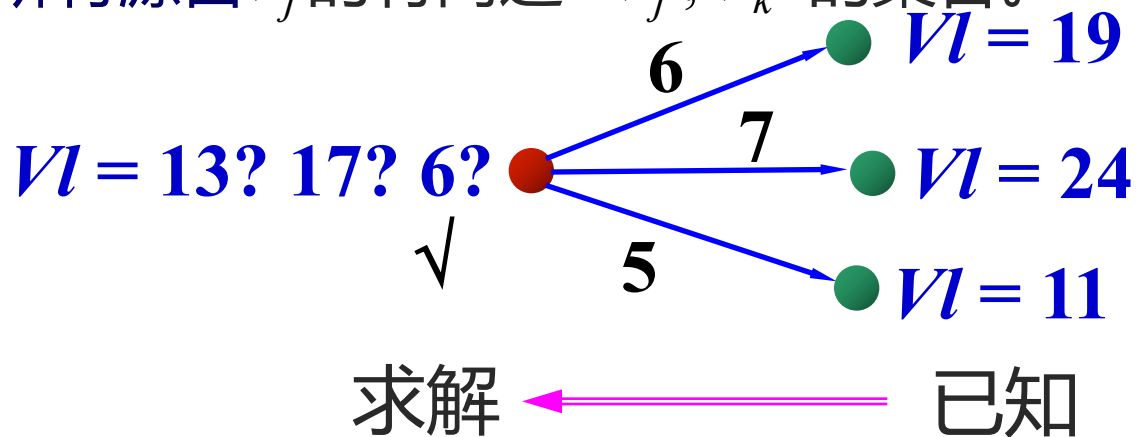


- 从 $VL[n-1] = Ve[n-1]$ 开始, 反向递推

$$VL[j] = \min_k \{ VL[k] - dur(< V_j, V_k >) \},$$

$$< V_j, V_k > \in S1, j = n-2, n-3, \dots, 0$$

$S1$ 是所有源自 V_j 的有向边 $< V_j, V_k >$ 的集合。



- 这两个递推公式的计算必须分别在拓扑有序及逆拓扑有序的前提下进行。



- 设活动 a_k ($k=1, 2, \dots, e$)在带权有向边 $\langle V_i, V_j \rangle$ 上, 其持续时间用 $dur(\langle V_i, V_j \rangle)$ 表示, 则有

$$Ae[k] = Ve[i];$$

$$Al[k] = Vl[j] - dur(\langle V_i, V_j \rangle); \quad k = 1, 2, \dots, e。$$

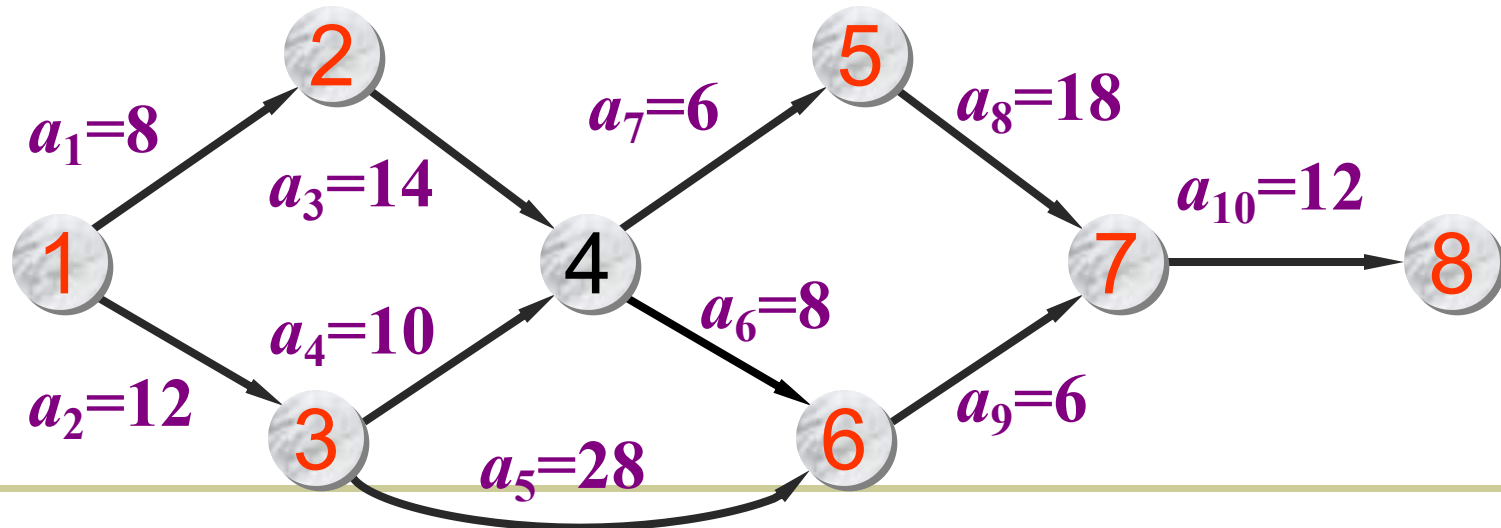
这样就得到计算关键路径的算法。

- 为了简化算法, 假定在求关键路径之前已经对各顶点实现了拓扑排序, 并按**拓扑有序的顺序**对各顶点重新进行了**编号**。



	1	2	3	4	5	6	7	8
<i>Ve</i>	0	8	12	22	28	40	46	58
<i>VI</i>	0	8	12	22	28	40	46	58

	1	2	3	4	5	6	7	8	9	10
<i>Ae</i>	0	0	8	12	12	22	22	28	40	46
<i>Al</i>	0	0	8	12	12	32	22	28	40	46





利用关键路径法求AOE网的 各关键活动



```
template <class T, class E>
void CriticalPath(Graph<T, E>& G) {
    int i, j, k;  E Ae, Al, dur;
    int n = G.NumberOfVertices();
    E *Ve = new E[n]; E *Vl = new E[n];
    for (i = 0; i < n; i++) Ve[i] = 0;
        for (i = 0; i < n; i++) { //正向计算Ve[], 假设已经
                                按拓扑排序从0到n-1编号
                j = G.getFirstNeighbor(i);
                while (j != -1) {
                    dur = G.getWeight (i, j);
```



```
        if (Ve[i]+dur > Ve[j]) Ve[j] = Ve[i]+dur;  
        j = G.getNextNeighbor(i, j);  
    }  
}  
Vl[n-1] = Ve[n-1];  
for (j = n-2; j > 0; j--) {    //逆向计算Vl[]  
    k = G.getFirstNeighbor (j);  
    while (k != -1) {  
        dur = G.getWeight (j, k);  
        if (Vl[k]-dur < Vl[j]) Vl[j] = Vl[k]-dur;  
        k = G.getNextNeighbor (j, k);  
    }  
}
```



```
for (i = 0; i < n; i++) {                                     //求各活动的 $A_e, A_l$ 
    j = G.getFirstNeighbor (i);
    while (j != -1) {
        Ae = Ve[i]; Al = Vl[j]-G.getWeight(i, j);
        if (Al == Ae)
            cout << "<" << G.getValue(i) << ","
                << G.getValue(j) << ">"
                << "是关键活动" << endl;
        j = G.getNextNeighbor (i, j);
    }
}
delete [] Ve; delete [] Vl;
};
```



注意



- 所有顶点按拓扑有序的次序编号。
- 仅计算 $Ve[i]$ 和 $VL[i]$ 是不够的，还须计算 $Ae[k]$ 和 $AL[k]$ 。
- 不是任一关键活动加速一定能使整个工程提前。想使整个工程提前，要考虑各个关键路径上所有关键活动。
- 如果任一关键活动延迟，整个工程就要延迟。



第8章书面作业



提交时间：12月11日，上课前提交

- (1) 第392页，8.1
- (2) 第392页，8.10
- (3) 第392页，8.11
- (4) 第393页，8.17